

HelixQA Autonomous Vision QA Guide

Field	Value
Revision	1
Created	2026-06-08
Last modified	2026-06-08
Status	active
Issues	LVA-009 (launch-dispatch exit-127)
Issues summary	one open known issue — see §6 Troubleshooting
Fixed	claude-CLI bridge 3-layer incompatibility (--json, --image, prompt position)
Continuation	docs/CONTINUATION.md

This guide documents the HelixQA autonomous vision QA path as it actually works in this repository today: Claude vision drives the real Lava Android app running on a Genymotion VM, analysing live screenshots and deciding ADB actions. It is accurate to the source files cited in §8; it makes no aspirational claims. One known issue (LVA-009) is open and documented as such.

Table of contents

- [1. Overview](#)
- [2. Prerequisites](#)
- [3. How the claude-CLI bridge works](#)
- [4. Running a bank](#)
- [5. Writing a new bank](#)
- [6. Troubleshooting](#)
- [7. Anti-bluff posture](#)
- [8. Sources cited](#)

1. Overview

The HelixQA autonomous vision QA replaces manual exploratory testing of the Lava Android app with a Claude-vision agent that drives the real app on a real (virtualised) device:

1. HelixQA takes a screenshot of the live device over ADB.
2. The screenshot path is handed to the `claude` CLI (the vision bridge), which reads and analyses the image and returns a single next action (tap, key, back, home, text, launch, shell, or noop).
3. HelixQA dispatches that action against the device over ADB and captures a new screenshot.
4. The loop repeats (up to a bounded step count) until the model reports the per-case goal reached or the step budget is exhausted.

Every test case comes from a YAML **bank**. Each case declares a launch step, `required_evidence` (screen-visible substrings the model confirms), and an `expected_result`. The orchestrator turns those into a `visionnav.Target{LaunchAction, ScreenGoals}` and runs the vision loop — this wiring lives in `submodules/helixqa/pkg/orchestrator/definition_challenge.go` and `cmd/helixqa/main.go`'s `buildAndroidVisionContext`.

Relationship to the §6.AE / §6.G Challenge gate

The vision QA is **complementary** to, not a replacement for, the Compose UI Challenge Tests run via `connectedDebugAndroidTest` (the §6.AE per-feature Challenge mandate, §6.G end-to-end provider verification, gated through the Containers/Genymotion VM path in `scripts/run-genymotion-challenges.sh`):

- The Challenge Tests are deterministic instrumentation tests authored in Kotlin; they are the load-bearing acceptance gate that `scripts/tag.sh` consults.
- The vision QA is an autonomous, exploratory, screenshot-driven layer that exercises the same real provider journeys (onboard → select provider → search → browse → open item → obtain download) that the §6.G banks describe, but driven by a vision model rather than hard-coded test code. It surfaces user-visible breakage the deterministic suite may not assert against, in the §6.J / §6.L "tests pass but feature doesn't work for users" spirit.

Both run the emulator inside a VM (Genymotion) per §6.AH — never host-direct, never a live ADB device that other projects are using.

2. Prerequisites

Prerequisite	Why	How
A Genymotion VM (or container/VM per §6.AH) running Android, reachable over ADB	§6.AH forbids host-direct emulators; the vision loop needs a real device serial to drive	Boot a Genymotion device; confirm <code>adb devices</code> lists its serial (the evidence runs used <code>127.0.0.1:6555</code>)
claude CLI on PATH	It IS the vision bridge — the only vision-capable provider HelixQA wires today	<code>buildAndroidVisionContext</code> calls <code>osexec.LookPath("claude");</code> if absent, android cases honestly SKIP (never a fake PASS)
Lava debug app installed on the VM	The banks launch <code>digital.vasic.lava.client.dev</code>	Install via <code>./gradlew :app:installDebug</code> — NOT a raw <code>adb install</code> (the §6.X guard hook gates raw installs; the Gradle task is the sanctioned path)
Tracker/RUTOR credentials in <code>.env</code> (form-login banks only)	Form-login providers (RuTor, RuTracker, Kinozal, NNM-Club) need real credentials to complete their journey; <code>archiveorg/gutenberg</code> are <code>AuthType.NONE</code> and need none	Populate the <code>gitignored.env</code> per §6.H — never commit credentials
The <code>helixqa</code> binary built	The runnable QA driver	<code>cd submodules/helixqa && go build -o bin/helixqa ./cmd/helixqa</code>

Optional (not required for a working vision run):

- `HELIX_TESSERACT_URL` — when set, an OCR-backed explorer captures an `OCRSnapshot` and runs the §11.4.52 OCR goal-match alongside the vision path. When unset, goal detection comes purely from the vision provider's own `Decision.GoalReached` (the model sees each

screenshot and confirms goal reached) — this is what lets the vision run work with no OCR infra.

- `HELIX_VISION_MAX_STEPS` — overrides the default 12-step-per-case budget.

3. How the claude-CLI bridge works

The bridge is `submodules/helixqa/pkg/llm/bridge_provider.go` (`BridgedCLIProvider`). It shells out to the `claude` CLI in non-interactive print mode and parses the JSON response. Three invocation specifics are load-bearing — each was a real incompatibility that was fixed this session, and each is documented in-source so a future maintainer does not regress it.

3.1 `--output-format json`, not `--json`

The argument list built in `buildArgs` is:

```
claude --print "<prompt>" --output-format json [--allowedTools Read] [--model <m>]
```

The current Claude CLI selects structured output via `--output-format json`. The obsolete `--json` flag is rejected with unknown option `'--json'`. The JSON response's primary content field is `result` (with `content / text` fallbacks handled in `parseResponse`).

3.2 No `--image` flag — the screenshot path is embedded in the prompt

The current Claude CLI has **no** `--image / --attach` option (it is rejected with unknown option `'--image'`). Instead, `Vision()`:

1. Writes the screenshot bytes to a temp `.png` file (absolute path).
2. Prepends a directive to the prompt via `buildVisionPrompt`:
"Here is the current screen. Read the screenshot image at `<abspath>` and analyse it. `<original prompt>`".
3. Passes `--allowedTools Read` so Claude Code's built-in **Read** tool reads and analyses the image regardless of the host's default tool-permission config.

`SupportsVision()` returns true only for `cliName == "claude"`. The path **MUST** be absolute (the `Read` tool needs it); `os.CreateTemp` already returns an absolute path and `Vision()` absolutizes defensively.

3.3 Prompt position — before the variadic --allowedTools

This is the subtle trap. The prompt **MUST** be passed as the **value immediately after --print**, NOT as a trailing positional argument:

```
GOOD: claude --print "<prompt>" --output-format json --
allowedTools Read
BAD:  claude --print --output-format json --allowedTools Read
"<prompt>"
```

--allowedTools is **variadic**: if the prompt trails it, commander.js consumes the prompt as just another tool name, leaving claude with no prompt at all (Input must be provided either through stdin or as a prompt argument when using --print). buildArgs therefore builds {"--print", prompt, "--output-format", "json"} first and only then appends the variadic --allowedTools Read. This was verified on the host and is recorded verbatim in the buildArgs comment.

4. Running a bank

The exact command (archiveorg journey example):

```
cd submodules/helixqa
./bin/helixqa run \
  --banks ../../lava-api-go/qa/banks/lava-archiveorg-journey.yaml \
  --platform android \
  --device 127.0.0.1:6555 \
  --package digital.vasic.lava.client.dev \
  --output <out> \
  --report markdown
```

Flag notes (from cmd/helixqa/main.go cmdRun):

- --banks is required (comma-separated files or directories).
- --device is the Android serial. Supplying it triggers buildAndroidVisionContext; on success the run prints Android vision backend: ENABLED (serial=...). If claude is not on PATH (or another prerequisite is missing) it prints Android vision backend: disabled (<reason>) – android cases will honestly SKIP.
- --package is the app package the banks launch (digital.vasic.lava.client.dev for the debug build).
- --report is markdown | html | json (default markdown).
- --output is the results/evidence directory (default qa-results).
- Other defaults: --validate (crash detection) on, --record on, --tickets on, --timeout 30m, --speed normal.

Where evidence lands

- The report is written under the run's `--output` directory; its path is printed as `Report: <path>` at the end of the run.
- Per-device vision evidence (every screenshot + the provider's recorded rationale, for §11.4.83 replay) is persisted under `<output>/vision-evidence/<serial>/` via the `FileSink`.
- In this repository, completed VM QA runs are archived under `.lava-ci-evidence/helixqa/<run-id>/` — e.g. `.lava-ci-evidence/helixqa/vm-qa-vision2-20260608T182251Z/` holds `qa-report.md` and `run.log`.

How to read the verdict

Each vision case produces two assertions in the report — these are the real anti-bluff signals:

Assertion	Meaning
<code>vision-screen-changed</code> (<code>session.ScreenChanged</code>)	The dispatched actions produced an observable change on screen. If false, the case FAILs with zero-screen-delta across all steps (actions produced no observable change – goal match is unearned) even if the model claimed the goal was reached.
<code>vision-goal-reached</code> (<code>session.GoalReached</code>)	The vision model confirmed the case's <code>expected_result</code> / <code>required_evidence</code> goal from a real screenshot.

A case PASSes only when both hold. A `vision-goal-reached: PASS` with a `vision-screen-changed: FAIL` is explicitly treated as an **unearned** goal match — see §7.

5. Writing a new bank

Mirror `lava-api-go/qa/banks/lava-archiveorg-journey.yaml`. Banks live under `lava-api-go/qa/banks/lava-*-journey.yaml`. Top-level shape:

```
version: "1.0"
name: "Lava – <Provider> Provider Journey"
description: "End-to-end <provider> journey on the Lava Android
  app: ..."
metadata:
  author: "vasic-digital"
```

```

app: "Lava"
application_id: "digital.vasic.lava.client.dev"
provider_id: "<provider-id>"
provider_display_name: "<Display Name>"
auth_type: "NONE | FORM_LOGIN | API_KEY"

test_cases:
- id: LAVA-<PROVIDER>-001          # per-case id convention:
    LAVA-<PROVIDER>-NNN
  name: "Onboard and select the <Provider> provider"
  category: functional
  priority: critical
  platforms: [android]           # routes the case to the
    android vision backend
  dispatches_to: "monkey -p digital.vasic.lava.client.dev 1"
  steps:
    - name: "Launch Lava and reach onboarding"
      action: "adb_shell: monkey -p
        digital.vasic.lava.client.dev 1"
      expected: "Onboarding Welcome screen is visible without a
        crash"
    - name: "Advance to provider selection"
      action: "Tap the primary continue control to reach
        provider configuration"
      expected: "Provider list shows <Provider> as a selectable
        entry"
  required_evidence:             # become OCR/vision goal
    substrings the model confirms
    - "<Display Name>"
    - "provider"
  tags: [<provider-id>, onboarding, provider-selection]
  expected_result: "The provider list is on screen and
    <Provider> is selectable"

```

Required structural rules (enforced by pkg/orchestrator/definition_challenge.go):

- **The launch step is mandatory and must come first.** The launch action (deriveLaunchAction) is the first non-skipped adb_shell: / shell: step. Every case opens with a step that monkey-starts the app, so the derived launch action is shell monkey -p digital.vasic.lava.client.dev 1 (non-empty — it must pass Target.Validate; without it the case is "not drivable").
- **required_evidence** entries plus expected_result become the ScreenGoals (OCR/vision goal substrings the model confirms from a real screenshot). Keep them short, screen-visible phrases a real user would actually see (e.g. "Search", "results", "Download", "Internet Archive").
- **expected_result** is the human-readable success criterion and the final screen goal.

- **platforms:** `[android]` routes the case through the android vision backend.
- **Per-case id** follows LAVA-<PROVIDER>-NNN (zero-padded, sequential), e.g. LAVA-ARCHIVEORG-001 ... -004.

For `AuthType.NONE` providers (archiveorg, gutenberg) include the anonymous "Continue without hanging on the loading indicator" regression case (the §6.G forensic anchor — the historical stuck-CircularProgressIndicator defect). For `FORM_LOGIN` providers the journey enters real credentials sourced from `.env` (§6.H — never inline them in the bank).

6. Troubleshooting

6.1 Bridge errors — all fixed (history kept as forensic record)

Symptom	Cause	Fix (landed)
unknown option '--json'	obsolete flag	use <code>--output-format json</code> (§3.1)
unknown option '--image'	the CLI has no image flag	embed the absolute screenshot path in the prompt + <code>--allowedTools Read</code> (§3.2)
Input must be provided either through stdin or as a prompt argument when using <code>--print</code>	prompt placed after the variadic <code>--allowedTools</code> , which consumed it as a tool name	pass the prompt as <code>--print's value</code> , before <code>--allowedTools</code> (§3.3)

All three are fixed in `pkg/llm/bridge_provider.go` and documented in-source. If any recurs, check `buildArgs / Vision / buildVisionPrompt` first.

6.2 Open: LVA-009 — launch-dispatch exit-127

Status: OPEN. In the latest VM run (`.lava-ci-evidence/helixqa/vm-qa-vision2-20260608T182251Z/qa-report.md`), the first case (LAVA-ARCHIVEORG-001) ERRORed with:

```
android vision run aborted: visionnav: step 2: dispatch
"launch digital.vasic.lava.client.dev": exit status 127
```

Exit status 127 is "command not found" from the dispatched launch action. The vision model correctly analysed the home-launcher screen and decided to launch the Lava client, but the dispatch of that launch action failed with 127. The remaining cases in that run then degraded to a stable LeakCanary screen with zero screen delta (the app's debug LeakCanary UI), so they FAILED on vision-screen-changed despite the model reporting vision-goal-reached — the unearned-goal guard correctly caught them. This launch-dispatch exit-127 is the single known remaining issue and is **not yet fixed**.

6.3 "Android vision backend: disabled — android cases will honestly SKIP"

A prerequisite is missing. The printed reason names which one (no claude on PATH, no device serial, evidence-sink error, etc.). The cases SKIP — they do **not** fake a PASS. Fix the named prerequisite (§2) and re-run.

6.4 vision-screen-changed: FAIL on every step

The dispatched actions produced no observable change (the run got stuck on one screen). Confirm the app actually launched (see LVA-009 above), that the correct package is installed via `./gradlew :app:installDebug`, and that the device serial is the live VM.

7. Anti-bluff posture

The vision QA is built to make a false PASS hard, per §6.J / §6.L and the HelixQA §107 / §11.4 anti-bluff covenant:

- **vision-screen-changed is the load-bearing real signal.** If the model claims the goal is reached but the screen never changed, the case FAILs with zero-screen-delta across all steps (actions produced no observable change – goal match is unearned). Goal match without observable change is treated as **unearned** — exactly the bluff this guard exists to catch.
- **vision-goal-reached comes from a real screenshot**, analysed by the vision model (or, when HELIX_TESSERACT_URL is set, cross-checked by OCR goal-match). It is never a metadata-only or config-only assertion.

- **Honest SKIP vs ERROR vs FAIL:**
 - **SKIP** — a prerequisite is genuinely absent (no claude, no device). The case does not run and is not counted as PASS. This is the honest "we could not test this" state.
 - **ERROR** — the run aborted on an infrastructure fault during the case (e.g. LVA-009's launch-dispatch exit-127). The case is a failure, not a PASS.
 - **FAIL** — the case ran but the user-visible goal was not genuinely reached (including the unearned-goal / zero-screen-delta case).
- **The report itself is the auditable proof.** `qa-report.md` records the per-challenge captured runtime output, the model's rationale, and the assertion ledger (`vision-goal-reached`, `vision-screen-changed`) — the report IS the \$107.x / \$11.4.83 captured evidence that the command ran and emitted the asserted output, with every screenshot persisted under `vision-evidence/<serial>/`.

A green vision run means a vision agent genuinely drove the real app on a real VM and observed the user-visible journey complete. CI green is necessary, never sufficient.

8. Sources cited

This guide was written against the live source in this repository (\$11.4.99 latest-source discipline). The cited files:

- `/Volumes/T7/Projects/Lava/submodules/helixqa/pkg/llm/bridge_provider.go` — the claude-CLI bridge: `buildArgs`, `Vision`, `buildVisionPrompt`, `SupportsVision`, `parseResponse` (the three \$3 invocation specifics).
- `/Volumes/T7/Projects/Lava/submodules/helixqa/cmd/helixqa/main.go` — the run command flag surface (`cmdRun`) and `buildAndroidVisionContext` (the vision-backend wiring + honest-SKIP behaviour).
- `/Volumes/T7/Projects/Lava/lava-api-go/qa/banks/lava-archiveorg-journey.yaml` — the bank schema, `launch-step` / `required_evidence` / `expected_result` contract, and the per-case id convention.
- `/Volumes/T7/Projects/Lava/.lava-ci-evidence/helixqa/vm-qa-vision2-20260608T182251Z/qa-report.md` — the real run evidence: the `vision-goal-reached` / `vision-screen-changed` assertions and the LVA-009 launch-dispatch exit-127 ERROR.
- `/Volumes/T7/Projects/Lava/scripts/run-genymotion-challenges.sh` — the \$6.AH VM Challenge path and `./gradlew :app:installDebug` / Containers-driven Genymotion context the vision QA shares.

Sources verified

Sources verified 2026-06-08 (in-repo source, not external docs): the five files listed in §8, read directly at the revisions present in the working tree this session.